Exercises for
Introduction to Machine Learning
Linda-Sophie Schneider, Utkarsh Bansal, Md Abdul Ahad,
Keyi Hou, Max Zernickel, Yumeng Zhang
Task 5: deadline: 12th July 2026

Pattern Recognition Lab

Exercise 05: KNN, NBNN and Logistic Regression on Kaktovik Symbols

In this task, you will implement and evaluate a small image-classification pipeline on the Kaktovik symbol dataset. The focus is on direct classification in pixel space using nearest-neighbour methods and a linear baseline classifier.

You will work with the Kaktovik images in `images/train`, `images/test`, and `images/unit_test`. The main goals are:

- implement a K-Nearest Neighbors classifier from scratch,
- implement a Naive Bayes Nearest Neighbor (NBNN) classifier from scratch,
- implement a confusion matrix using NumPy only,
- compare Euclidean KNN, cosine KNN, Euclidean NBNN, cosine NBNN, and `LogisticRegression` from sklearn on the same dataset,
- and generate plots that document the behaviour of the classifiers.

Hint: This exercise connects directly to the lecture material on nearest-neighbour classifiers, Naive Bayes Nearest Neighbor, distance measures, confusion matrices, and logistic regression. Please first review the relevant lecture slides before you start coding.

Exercise 1

Read first

The provided Python files already contain most of the exercise structure. Read the files first to understand:

- how the Kaktovik images are stored,
- that the image loading code is already provided,
- which parts are support code,
- and which functions you are expected to implement yourself.

Do you need to write the data loader yourself?
No. The function `load_images_labels()` in `main.py` is already provided and should be used as it is. The exercise focuses on the classifiers, the confusion matrix, and the evaluation plots, not on implementing file I/O.

Rules on imports and helper functions
Use the provided file structure as it is.

- In `knn.py`, `nbnn.py`, `logistic_regression.py`, and `evaluation.py`, do not add new imports or new helper functions from other local files.
- In `knn.py` and `nbnn.py`, use NumPy only; do not use pre-built nearest-neighbour or distance helpers.

- In `logistic_regression.py`, you may use NumPy and `sklearn.linear_model.LogisticRegression`; do not replace it by a different model or add extra model-selection tools.
- In `evaluation.py`, use NumPy only; do not use a pre-built confusion matrix implementation.
- In `main.py`, use the provided loading, plotting, and evaluation pipeline; do not rewrite the data loader or replace the required classifier comparison.

Exercise 2 Classifier implementations

Implement the three classifier modules

- `knn.py`,
- `nbnn.py`,
- `logistic_regression.py`.

The goal is to build a small common classification interface for all three approaches:

- a K-Nearest Neighbors classifier,
- a Naive Bayes Nearest Neighbor classifier,
- and a wrapper around `LogisticRegression` from sklearn.

All three should work on the same image representation and should be usable from the provided `main.py`. For KNN and NBNN, support both Euclidean and cosine distance. For KNN, use the provided visualization helper to create neighbour plots for the `unit_test` images.

You should think carefully about the required class interface, the handling of single samples versus batches, and the behaviour in common edge cases.

Exercise 3 `evaluation.py`

Implement the function `confusion_matrix(y_true, y_pred)` using NumPy only. It should count how often each true class is predicted as each output class and return the result as a square matrix. **Pay attention to the input shapes and the class indexing.**

Exercise 4 Main.py and Evaluation

Use the provided `main.py` to evaluate your implementation on the Kaktovik images. The script loads the local dataset, resizes the images, encodes the labels, trains the classifiers, generates neighbour visualizations, computes confusion matrices, and compares the final accuracies.

Compare:

- KNN vs. NBNN vs. `LogisticRegression`
- Euclidean distance vs. cosine distance for KNN and NBNN

For the quantitative evaluation, use a **very large part of the dataset**: in the provided setup, this means the full official test split in `images/test`. Do not restrict the comparison to only a few hand-picked examples. The folder `images/unit_test` is reserved for the 11 neighbour visualizations only.

Your output should include:

- 11 nearest-neighbour plots for the `unit_test` set for KNN,

- confusion matrix plots for all classifier variants,
- and one accuracy comparison plot.

This means that all three classifier families should appear in the final evaluation material:

- KNN with neighbour visualizations and confusion matrices,
- NBNN with confusion matrices,
- Logistic Regression with confusion matrices.

The intended split of responsibilities is therefore:

- `images/train`: train all classifiers,
- `images/test`: compute accuracies and confusion matrices on a large evaluation set,
- `images/unit_test`: generate the 11 KNN neighbour plots.

Include your code and the generated plots in your submission.

Exercise 5 Short Discussion

Write a short comment on one observation you made. For example:

- Did Euclidean and cosine distance behave similarly?
- Did KNN and NBNN differ noticeably on the Kaktovik symbols?
- Which classifier worked best on the large test split?